

第 1 章 初识 AI Infra

读者可能会问，为什么要学习 AI Infra？我们不妨先想一想：大多数软件工程师不需要开发操作系统、编译器和芯片，为什么还要学习操作系统、编译原理和计算机体系结构？因为这些知识能帮助我们理解，程序所依赖的各种抽象是怎样实现的。抽象是把复杂的实现封装在明确的接口之后，让开发者用较少的概念组织程序。例如，程序可以申请一块内存、读取一个文件或调用一个函数；这些操作背后涉及资源管理、设备访问和指令执行。掌握它们的原理，开发者才能理解程序的行为、判断性能，并选择合适的实现方法。

基于模型开发应用，同样需要理解所依赖的平台。开发 Agent 与开发传统软件有一个重要区别：许多原来需要用代码逐条规定的行为，现在可以交给模型根据任务指令和上下文决定。开发者因此还需要选择模型、组织输入、提供工具，并安排多轮调用。我的上一本著作《深入理解 AI Agent》就是讲 Agent 架构设计与工程实践的。

《深入理解 AI Agent》的读者给我提出了很多问题。在解答这些问题的过程中，我愈发意识到，要开发好 Agent，需要两方面的基础知识。一方面是模型本身：模型怎样理解输入，如何生成输出，如何与环境交互。另一方面是模型的运行系统：参数和上下文状态存在哪里，计算怎样执行，多台设备怎样协作，以及模型调用怎样与工具程序衔接。理解模型类似于理解传统应用所依赖的操作系统；理解 **AI Infra** 则类似于学习计算机体系结构，认识支撑 Agent 的执行系统与资源代价。

本书研究第二方面：**AI Infra**。AI Infra 是支撑 AI 训练和推理的基础设施，包括计算设备、存储与互联，以及组织这些资源的软件。训练通过样本调整模型参数；推理使用训练得到的参数处理输入、产生输出。本书围绕这些过程讲解计算、存储、通信与调度。读者可以据此开发和部署模型应用，也可以进一步设计支撑这些应用的系统。

本章先说明应用的开发方式发生了什么变化，以及这种变化为什么值得单独研究。随后跟随一次请求的处理过程认识系统，再用容量、算力和带宽回答具体的设计问题：数据能否放下，回答要等多久，增加设备或改变数据表示能带来多少收益。最后通过真实架构的形成过程，理解需求如何推动系统设计。

1.1 AI Infra 为什么重要

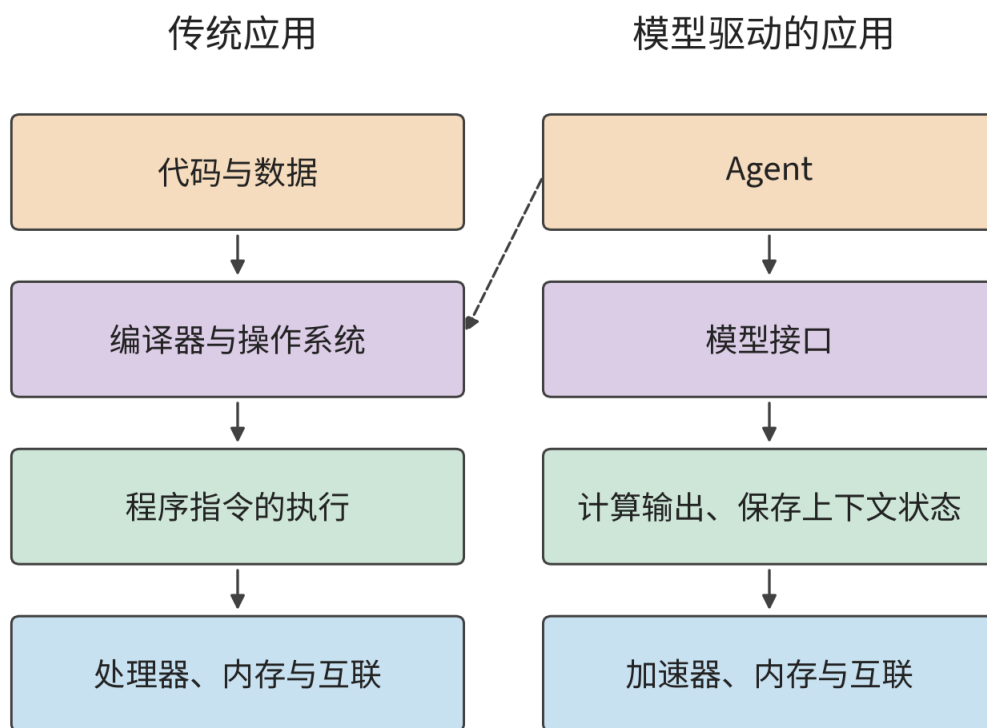
1.1.1 编程抽象上移：从操作系统到模型上下文

以一个文档问答服务为例。开发者先规定文档读取、检索和结果展示的程序流程，再把用户问题与检索到的材料交给语言模型生成答案。这里的上下文是本次模型调用可以使用的输入内容，包括任务指令、参考材料和此前的交互结果。保持模型参数不变，改变指令、示例或参考材料，就能改变它处理的任务及回答方式。

如果要把这个服务检查代码，可以把代码、测试要求和可以调用的工具一同提供给模型，由模型提出修改方案或发起工具调用；应用程序执行测试，把结果加入下一次调用的上下文，再由模型继续判断。工具是为应用调用的程序功能，例如读取文件或运行测试。模型负责选择下一步做什么，宿主程序负责检查权限、执行操作和保存结果。完成一次任务可能需要多次模型调用和工具执行。

可编程性是开发者通过某种接口改变系统行为的能力。传统应用主要用程序代码表达行为，编译器将代码转换为可执行指令，操作系统管理程序运行所需的资源。模型驱动的应用增加了另一个重要入口：通过选择模型、组织上下文和提供工具来表达任务。一部分原来需要逐条编写的行为规则，现在由模型根据上下文决定；代码继续组织调用流程，工具程序继续通过操作系统访问文件、网络和设备。¹⁰

图 1-1 对照了这两种组织方式。右侧的 Agent 负责组织上下文、调用模型，并执行模型选择的工具。右侧的模型接口是应用向模型提交输入、取得输出的调用方式。接口背后的运行系统计算模型输出，并保存后续步骤需要复用的上下文状态；加速器是适合并行执行大量模型运算的处理器，内存保存参数与运行状态，互联在设备之间传递数据。开发者从上层改变任务，底层要把这些变化落实为具体运算与数据访问。



工具程序仍由操作系统执行

图 1-1 应用行为的表达方式与执行路径。左侧通过代码组织程序，右侧通过上下文调用模型；实线表示向下的执行依赖，虚线表示工具执行仍需传统操作系统。加速器、内存与互联分别承担模型计算、数据保存和设备间传输的职责。

在 AI 系统中，同一个模型服务于不同应用时，会使用同一组参数完成各请求的计算，系统可以把多条请求放到一起，共同使用读入的参数，提高设备利用率；这称为批处理，每批共同执行的请求数称为批次大小（batch size）。与此同时，长文档、长回答和多轮工具调用会改变计算量、状态占用和等待时间。同一个模型、同一套设备，处理不同任务时，性能可能相差很大。

从模型设计来看，压缩保存的状态、改变注意力机制或只使用部分模型参数，可以改变硬件需要完成的工作；从硬件条件来看，存储容量、数据传输速度和设备连接方式，又会影响哪些模型结构和执行方法更合适。理解这种双向关系，才能同时讨论回答质量、响应时间和成本，并解释为什么某个应用的最佳方案在另一个应用中需要重新选择。

编程抽象从操作系统到模型上下文的迁移是计算机系统领域数十年来最重要的变化之一。计算机系统领域的两大顶级会议 SOSP（操作系统原理研讨会）、OSDI（操作系统设计与实现）中都包括“OS”（操作系统）。传统操作系统需要支撑多种多样的应用，编译器和硬件也要为事先未知的程序提供通

用能力。各层通过接口分工，在简化编程、隔离执行和提高性能之间作出设计取舍。例如，独立算子通过内存交换结果，便于组合成不同程序，也增加了中间数据的读写；主机统一提交设备任务，便于控制执行，也增加了反复交接的时间。

在以模型训练和推理为主要应用的系统中，各层可以围绕同一个完整的执行过程共同设计。模型结构给出计算和数据依赖，运行时掌握状态的产生与使用，硬件承担相应的运算和传输。利用这些信息，可以重新选择算子边界、存储表示、调度方式和设备交接路径，消除通用实现中的部分开销。同一模型又能通过不同上下文承接多种任务，既能支持多样的应用行为，又能让底层针对模型执行进行专门优化。上层可编程性的变化，为跨层联合优化打开了新的空间。本书通过量化分析研究这些优化机会，解释原有设计取舍如何改变，以及新的实现怎样提高训练和推理的性能。

1.1.2 从任务到设备的六层分工

先看 AI 系统怎样处理一次请求。应用提出回答质量和响应时间要求，模型按照自身结构对输入执行一组运算，服务系统安排这些运算在设备上执行。设备保存的数据、完成的计算和交换的结果，共同构成 AI Infra 的工作。图 1-2 按这一过程划分六个层次；后面的估算将逐层追踪同一个请求。

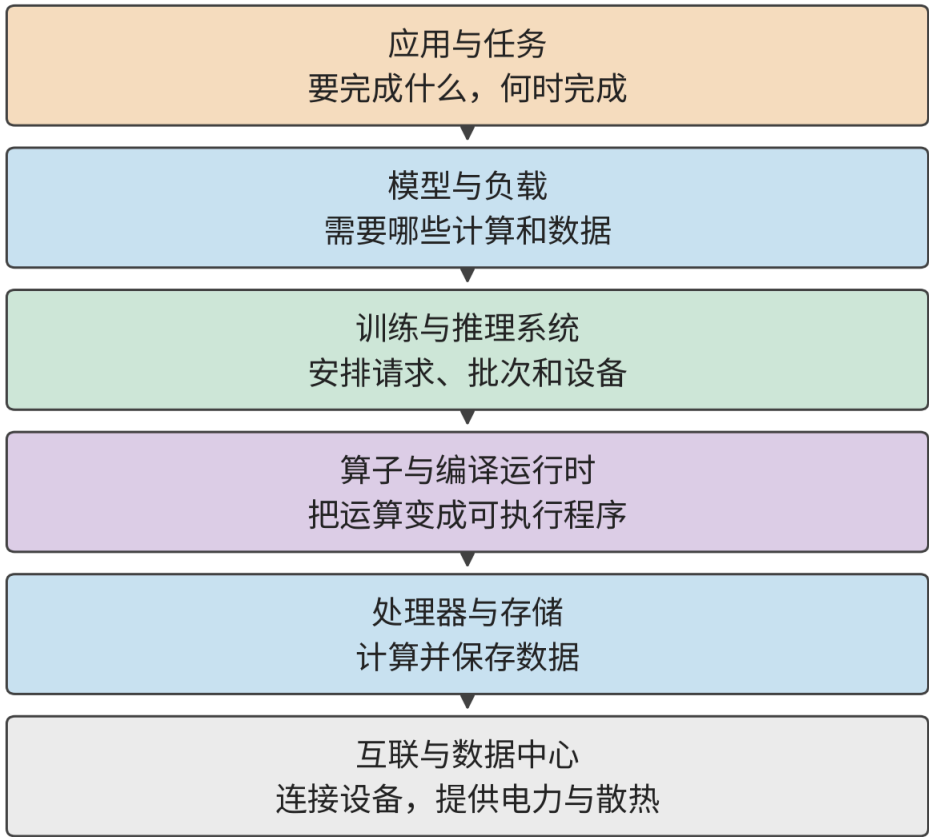


图 1-2 从应用到设备的六层分工。沿箭头向下，任务逐步落实为运算、程序和物理资源；调度、观测等跨层能力由正文说明。

最上面是应用与任务。对话、代码生成、语音交互，以及能够调用工具推进任务的智能体（Agent），都在这一层提出具体要求：要完成什么任务，多快返回结果，成本上限是多少。训练也有自己的任务目标，例如在给定时间内完成一次模型更新或一轮预训练。

向下一层是模型与负载。模型结构规定要进行哪些计算、保留哪些数据；实际负载则决定这些工作何时到来、持续多久。一个简短的问题和一篇很长的文档，即使交给同一个模型，也会形成不同的计算与存储需求。模型结构在第 2 章展开，训练与推理负载在第 3 章展开。

训练与推理系统负责把这些需求组织成可执行的工作。它接收请求或作业，安排批次，管理运行状态，并决定怎样使用一台或多台设备。在推理中，它要协调正在生成的请求和新进入的请求；在训练中，它还要组织参数更新、通信和恢复。第 8 至第 10 章讨论这一层。

再向下是算子与编译运行时。模型中的矩阵乘法、归一化（按统计量调整数值尺度）等操作，需要转换为处理器能够执行的程序。这里把矩阵乘法等基本运算称为算子；算子库提供这些运算的实现，编译器把程序转换为设备可执行的形式，运行时负责提交程序和管理执行中的资源。三者共同决定怎样分块、怎样使用临时存储、何时提交工作。相同的数学表达式，可以有不同的数据访问和执行方式，第 5 章会具体分析这些差异。

处理器与存储负责执行计算、保存数据。CPU 是执行通用程序的中央处理器；GPU 是擅长并行处理大量相似运算的图形处理器，如今也广泛用于模型计算；NPU 是面向神经网络运算设计的处理器。主存保存 CPU 使用的数据，显存保存 GPU 使用的数据，片上存储则位于处理器芯片内部，让计算单元能就近读取数据。计算单元先读取输入，再执行运算，最后保存结果。第 4 章将从计算能力与数据读取速度的关系出发，解释加速器的设计。

最下面是互联与数据中心。设备之间通过互联交换数据，服务器和更大的协作组通过网络连接，供电与散热决定资源能够怎样部署。第 6、7 章讨论超节点与数据中心网络，第 12 章进一步把执行位置扩展到端、边和云。

六个层次说明了主要分工，但有些能力贯穿多个层次。资源调度、运行环境、观测和计费需要结合多个层次的信息来组织；工艺、封装、供电和散热则决定设备能够怎样制造和部署。网络也并非只在最下方发挥作用：需要跨设备协作的模型、算子和服务，都要经过具体互联。

沿这六层向下看，任务逐步变成具体的计算与数据访问；向上看，设备容量、算力与带宽决定系统能运行多大的模型、同时处理多少请求，以及多快作出响应。海思首席科学家廖恒博士提出的“十八层宝

塔”，用来描述从应用、模型与软件系统，到芯片、制造工艺和底层物理的多个层次，强调这些层次之间的联系。本书据此组织全栈视角，并按教学需要将讨论内容归为图 1-2 的六层。¹

这些层次也标出了可以共同优化的边界。模型改变状态表示，会改变存储和通信需求；编译器将相邻算子融合，会改变中间结果的保存位置；运行时让设备直接交接，会改变主机参与的次数。后续章节将从这些具体变化出发，计算省下的读写、提交和等待，再回到完整请求，判断哪一种组织方式更合适。

AI Infra 建立在云数据中心已有的服务器、网络、存储和资源管理技术之上，但模型执行的需求改变了资源配置的重点。同一个模型可以承接问答、代码生成等多种应用；一次模型调用又可能分布在许多加速器上，使网络通信直接进入计算依赖链。基础设施既要服务独立请求，也要组织设备之间的紧密协作。

多个独立用户共用资源的服务称为多租户服务。它既要共享模型计算能力，也要隔离不同用户的数据和权限；第 11 章将讨论相应的运行环境。下面沿一次生成请求，把六层分工连接起来。

1.1.3 一次生成请求的处理过程

服务实例是能够独立完成模型请求的一组执行资源，可以由一张或多张设备组成。一次请求由其中一个实例承接。图 1-3 展示了请求分配与实例内部执行的分工：服务路由器是选择请求执行位置的软件，选定实例后，由实例内部安排设备执行模型。先分清谁负责执行，才能知道一份权重应放在哪里、一次中间结果应传给谁。

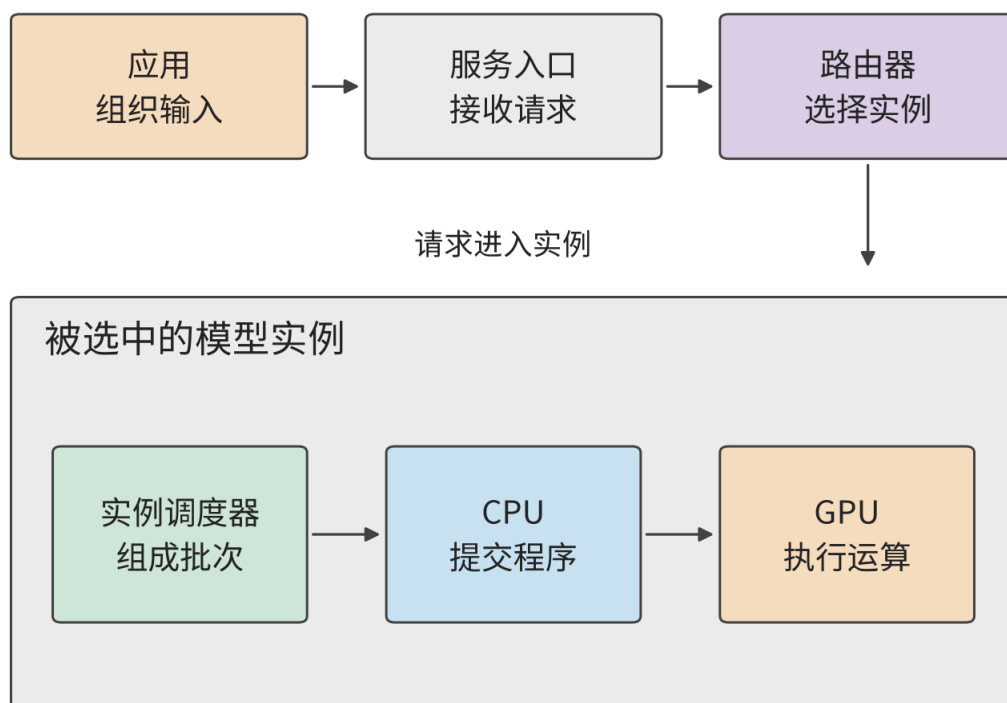


图 1-3 一次请求先由路由器选择模型实例，再由实例内部的调度器安排执行。灰色外框圈出同一个实例，实线表示请求和工作提交的方向。

上下文（context）是模型当前可使用的输入信息，包括用户输入、已有对话、检索结果和此前生成的内容。应用将这些信息组织成请求，交给模型服务入口。入口核验访问权限、检查配额和请求内容后，路由器依据目标模型、实例负载等条件选择副本。这里的副本指一个能够独立完成请求的模型实例，可以运行在一张卡上，也可以分布在多张卡上。路由选定实例之后，实例内部再安排这些设备协作。

文本随后被转换成 token 序列。词元（token）是模型处理文本的基本单位，可能对应一个字、词的一部分或其他片段，具体文本的 token 数由分词结果确定。文本转换可以由实例或共享前端完成。实例调度器将请求放入队列，将请求编入批次，并为其状态分配空间。CPU 上的运行时再向 GPU 提交计算任务，GPU 执行相应算子。¹¹

模型先处理输入，这一阶段称为预填充（prefill）；随后逐步生成新 token，这一阶段称为解码（decode）。后续步骤会复用此前保存的状态，其中的 KV 缓存（KV cache）保存上下文位置产生的键和值两类向量，供后续位置计算注意力时复用。键和值是注意力机制使用的数据表示，第 2 章会从具体运算说明它们怎样产生。生成循环由实例内部的调度器持续推进，前一步输出成为下一步输入。产生的 token 经输出处理转换为文本，沿连接流式返回；如果输出要求调用工具，应用运行工具，把结果加入上下文，再发起后续调用。

模型权重是训练得到、用于把输入变换成输出的数值参数。它们在实例启动或模型切换时从存储加载，随后保留在显存中（称为驻留），供多次请求使用。生成时，GPU 计算单元从显存读取当前层的权重和上下文状态，完成运算，再将结果交给下一层。于是，同一份数据经历了两种不同频率的搬运：加载把模型送到设备，执行则反复把所需数据送到计算单元。在多卡实例中，一张卡算出的中间结果还要传给其他卡，后续计算才能继续。中间张量是运算产生并交给后续运算的多维数值数组，矩阵就是二维张量。外部请求可能只有一小段文字，内部搬运的数据却包括大得多的权重、状态和中间张量。

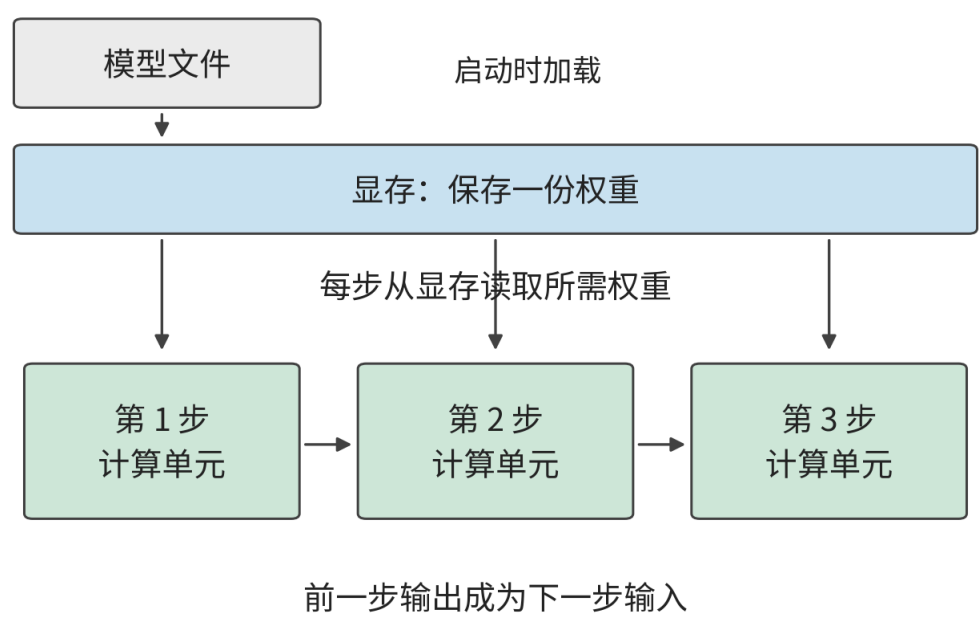


图 1-4 权重加载一次，生成时逐步读取。蓝框表示显存中持续保存的同一份权重，三个绿框表示先后发生的计算；箭头表示读取或结果依赖，步骤间距不代表耗时。

这些软件功能最终要在机房中的设备上运行，图 1-5 展示了它们对应的硬件。数据中心包含入口与 CPU 服务、共享存储，以及由多个超节点构成的加速器资源池。超节点是通过高带宽互联紧密协作的一组设备，可以跨多个服务器或计算托盘组织。

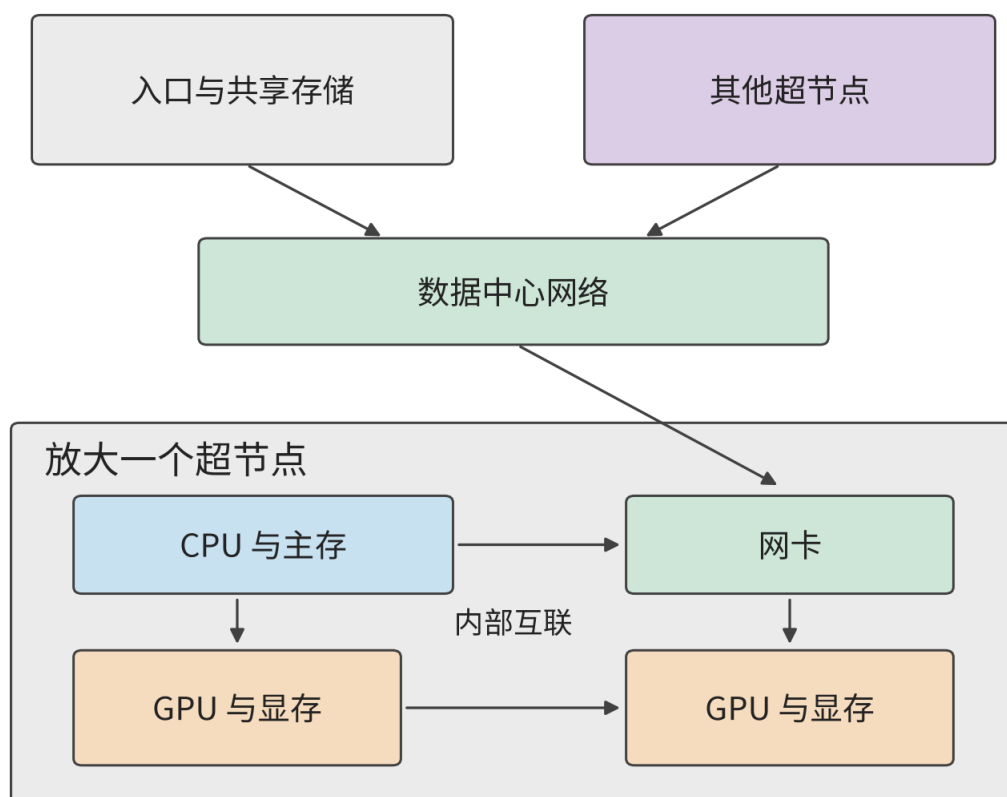


图 1-5 物理连接的两层视图。上方用数据中心网络连接服务与超节点，下方放大一个超节点，显示主机、网卡、GPU 与显存；线条展示数据路径，设备数量用于示意。

在一个服务器或计算托盘内，CPU 连接主存，负责主机程序和执行控制；GPU 连接各自的显存，承载模型计算与驻留数据。HBM（高带宽内存）是加速器常用的一类显存，通过宽接口持续提供数据。CPU 与 GPU 可以通过 PCIe（高速外设互联）连接，GPU 之间也可以通过独立的高速互联直接交换数据。网卡把本机连接到网络。在支持直接内存访问的系统中，数据可以由传输硬件直接写入指定内存区域，CPU 负责发起和管理传输；第 7 章会进一步解释这种交接。

超节点内部的互联通常称为 **scale-up**，重点是让一组设备以较低的通信开销共同计算；多个超节点再通过网卡与交换网络 **scale-out**，扩大集群范围。两者都要考虑带宽、延迟、拥塞与故障，但协作范围和代价不同。以 NVIDIA 的 GB200 NVL72 为一个具体例子，官方设计把 36 个 Grace CPU 和 72 个 Blackwell GPU 组织在一个机架中，72 个 GPU 通过 NVIDIA 的 GPU 高速互联 NVLink 直接协作，形成同一个互联域，集群网络则承担向外连接。这一组织把紧密协作范围从单机扩大到机架，使模型可以在更多设备之间分配权重和状态，同时通过机架内互联完成频繁交接。¹²

AI 网络与传统数据中心网络由此形成了分工。服务入口传递请求，共享存储提供模型文件，设备互联传递计算中的中间结果。三种流量的频率、大小与等待关系不同：模型文件可以提前加载，当前层的部分结果却必须及时到达，下一层才能继续。物理连接因此直接影响模型的执行时间。

练习 1-1 [延伸]：追踪一次请求的数据

沿图 1-3 至图 1-5，补画 KV 状态随输出增加的过程，再分别标出模型文件加载、一步权重读取、KV 追加和卡间结果传递。若回答从 100 个 token 增至 1000 个，哪些数据只加载一次，哪些访问会增加？若把一个实例复制成两个实例，权重容量与请求路由又怎样改变？

一次模型执行既受单卡能力影响，也受设备连接方式影响。单卡的算力和存储带宽决定本地计算与读写的耗时，多卡协作则增加数据交换和等待。下面先估算单卡执行，再将同样的方法用于后续章节的超节点和网络分析。

DeepSeek V4 到 V4.1 Flash 的演进提供了一条可以沿全书追踪的线索。模型团队从长上下文 Agent 的输入处理、KV 保存与迁移成本出发，改变了上下文表示、层间共享和缓存恢复方式。¹⁷ 我们将围绕同一类会话展开讨论：Agent 等待工具返回，恢复此前上下文，处理新增输入，再继续生成。第 2 章计算状态的存储需求，第 3 章分解负载，第 4 至第 7 章追踪设备执行和数据路径，第 8、9 章讨论怎样缓存状态、分配请求，最后回到完整任务。Qwen3 的基础算例继续用于建立计算方法，V4 / V4.1 用于检验模型与系统怎样共同改变。

1.2 系统设计的关键数字

1.2.1 用数量级判断方案

Jeff Dean 在 2009 年的一次演讲中列出了一张后来被广泛引用的表，标题是“Numbers Everyone Should Know”。表中既有缓存、内存的访问时间，也有磁盘和网络操作的时间。它提供了一种很实用的工作方式：在写程序或搭建系统之前，先估算主要操作需要多少时间和资源。²

下面完整列出原幻灯片的 12 项操作和数值。所有时间统一为 ns（纳秒，十亿分之一秒）；1 μ s（微秒）等于 1000 ns，1 ms（毫秒）等于 1000 μ s。数据单位中，1 byte（字节）等于 8 bit（比特）；bit/s 表示每秒传送的比特数，Gbit/s 中的 G 表示十亿。容量单位 KB、MB、GB、TB 分别表示 10^3 、

10⁶、10⁹、10¹² 字节；KiB、MiB、GiB 分别表示 2¹⁰、2²⁰、2³⁰ 字节。这组数字对应 2009 年的设备环境，用于理解不同操作之间的数量级关系。

缓存保存近期可能再次使用的数据；一级缓存通常比二级缓存更小、更靠近计算核心。分支预测是处理器对接下来执行哪条指令路径的预判，预测错误时要重新安排执行。线程是程序中独立调度和执行指令的单位；互斥锁限制多个线程同时访问同一份共享数据。磁盘寻道是机械磁盘移动磁头、定位目标磁道的过程。Zippy 是 Google 当时使用的压缩库。这些操作分别涉及计算、同步、存储与网络，因此差距可以跨越多个数量级。

操作	时间 (ns)
访问一级缓存	0.5
分支预测错误后的恢复	5
访问二级缓存	7
对互斥锁加锁或解锁	25
访问主存	100
用 Zippy 压缩 1K 字节	3,000
经 1 Gbit/s 网络发送 2K 字节	20,000
从主存连续读取 1 MB	250,000
同一数据中心内往返	500,000
磁盘寻道	10,000,000
从磁盘连续读取 1 MB	20,000,000
数据包从加州到荷兰再返回	150,000,000

取其中三个历史数字：主存访问约 100 ns，同一数据中心内的一次往返约 500,000 ns，磁盘寻道约 10,000,000 ns。将单位改写得直观一些，就是 0.1 μs、0.5 ms 和 10 ms。

按这组数字，一次磁盘寻道所需的时间约为一次主存访问的十万倍。对串行等待的请求，这种差距会直接积累到完成时间上。若请求必须先找到一块数据，才能确定下一次访问的位置，处理器在两次访问之间即使只做很少的计算，也要等待整个存储访问过程。

例如，设请求串行执行 20 次寻道，每次 10 ms，全部 CPU 计算为 1 ms，总时间就是 201 ms。把计算速度提高一倍，只能省下 0.5 ms；通过连续放置相关数据，将寻道次数从 20 次减为 10 次，则省下 100 ms。两种优化分别作用于计算与等待，收益的差距来自它们在原执行时间中的占比。

把时间分解后，优化方向就可以通过计算来判断：某一部分在原总时间中占比越大，缩短它越有可能改善整体。计算只占 1/201 的寻道例子说明，即使这一部分完全消失，总时间也只减少约 0.5%。

将上述关系推广，便得到总加速比的计算公式。若原执行时间中有比例 f 的部分能够加速 s 倍，其余工作与执行次序保持不变，则总加速比为：

$$\text{Speedup} = \frac{1}{(1 - f) + f/s}.$$

这就是 Amdahl 定律。即使把这部分加速到耗时可以忽略，总加速比也不超过 $1/(1 - f)$ 。前面的寻道例中，CPU 计算只占总时间的约 1/201；仅优化这部分，收益必然有限。当寻道次数减少时，被缩短的是占比最大的等待部分，因此它对总时间的作用远大于 CPU 加速。

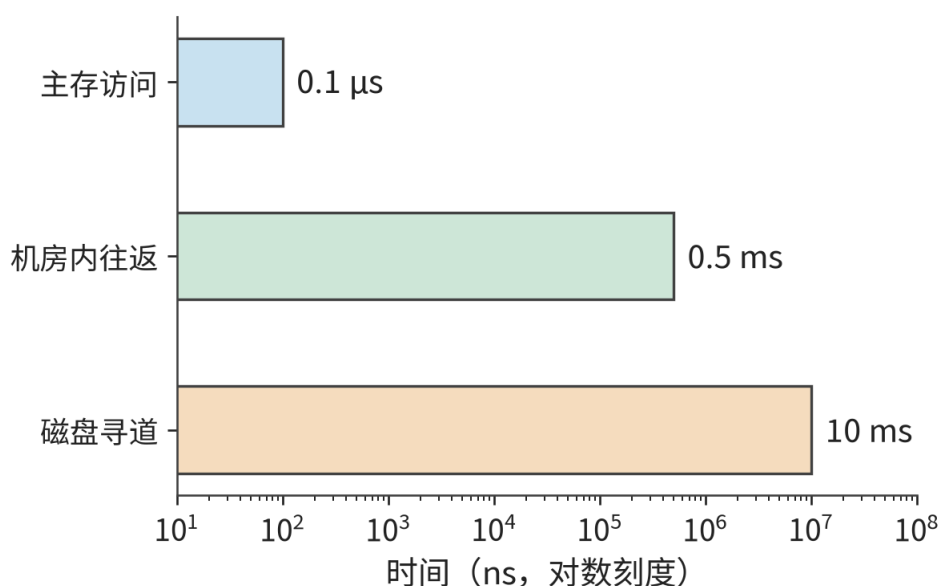


图 1-6 Jeff Dean 2009 年演讲中的三种操作延迟。横轴为对数刻度，每相邻数量级相差十倍；先识别操作，再比较它们在串行等待中的代价。

从这张历史资源表中学到的方法，可以直接用于模型执行：先确定各项工作使用什么资源，再比较它们所需的时间。接下来用加速器的容量、算力和带宽完成同样的估算。

1.2.2 分析 AI 系统需要哪些关键数字

设一个任务同时需要保存 M 字节，执行 F 次浮点运算，并通过某个存储接口读写 R 字节。设备提供容量 M_{cap} 、计算吞吐 Π 和接口带宽 β 。三个需求必须分别与相应资源比较：

$$M \leq M_{\text{cap}}, \quad T_{\text{compute}} = \frac{F}{\Pi}, \quad T_{\text{memory}} = \frac{R}{\beta}.$$

容量决定能否同时保存所需数据；后两项把工作量换算为时间。一份数据可以保存一次、读取多次，所以 M 与 R 要分别计算。若 Π 和 β 取峰值，得到的就是完成指定计算与读写所需的时间下界。

除了容量、计算吞吐和带宽，还需要认识操作延迟。下面分别说明这四类资源数字怎样进入估算。

容量回答能同时放下多少数据。例如，一张设备有多少显存，决定了它能否保存模型权重、上下文状态和运行时的临时空间。容量以 byte（字节）计量。

计算吞吐回答单位时间内能完成多少运算。常见的 FLOP 表示一次浮点运算，FLOPs 表示运算总次数，FLOP/s 表示每秒的浮点运算数。GFLOPs、TFLOPs 分别表示十亿、万亿次浮点运算，GFLOP/s、TFLOP/s 则表示相应的每秒速率。矩阵计算中一次乘法加一次加法通常计为两次运算。比较峰值时，还必须说明输入精度、累加精度以及是否使用稀疏条件。稠密计算按完整矩阵执行；稀疏计算利用零元素或规定的稀疏结构跳过部分运算。估算时间时，所用算力必须与计算条件一致。

带宽回答单位时间内能传送多少数据。显存带宽描述显存与芯片之间的数据搬运能力，卡间链路和网络带宽描述其他路径的能力。估算哪一段传输，就使用该段接口的带宽。

延迟回答发起一次操作以后需要等多久。高带宽接口也可能有不可忽略的启动或往返时间。传输很大的数据块时，耗时往往主要取决于数据量与带宽的比值；请求很小、又必须一次接一次等待时，启动和往返时间就可能占据主要部分。

H100 是 NVIDIA 的一代 GPU，SXM 指本例采用的模块形态。这里用 BF16 表示每个数占 16 位、即 2 字节的浮点格式，用 FP32 表示每个数占 32 位、即 4 字节的浮点格式；矩阵乘法可以读取 BF16 输入，再用 FP32 保存不断累加的结果。以 H100 SXM 为例，其显存容量为 80 GB，HBM 带宽为 3.35

TB/s，BF16 输入、FP32 累加的稠密矩阵峰值约为 989.4 TFLOP/s。下面用这三项规格估算容量、读取和计算时间。³

将 H100 与 NVIDIA 的 A100 80GB SXM、GeForce RTX 4090 放在一起，就得到一张 GPU 资源速查表。三者分别采用 Hopper、Ampere 和 Ada 架构。表中的容量使用厂商名义 GB，带宽使用十进制 TB/s；矩阵算力统一采用 BF16 输入、FP32 累加、稠密计算的峰值。最后一行由 1 GB 除以显存带宽得到。¹⁵

资源或操作	RTX 4090	A100 80GB SXM	H100 SXM
显存容量 (GB)	24	80	80
显存带宽 (TB/s)	1.008	2.039	3.35
BF16 矩阵峰值 (TFLOP/s)	165.2	312	989.4
按带宽峰值读取 1 GB (ms)	0.99	0.49	0.30

这张表提供了把工作量换算成时间所需的硬件性能数据。例如，读写量保持不变而带宽增加一倍， R/β 减半；计算量保持不变而算力增加一倍， F/Π 减半。接下来计算工作量，再比较计算和读写分别需要多久。

1.2.3 参数存储量与链路带宽的换算

这四类指标要与具体工作对应起来，才能用于估算。第一步是把模型参数换成存储字节数，判断设备能否放下；第二步再用字节数除以带宽，估算搬运这些数据需要多久。

以真实模型 **DeepSeek-R1-Distill-Llama-70B** 为例。它是基于 Llama 架构的稠密模型，公开权重包含约 705.54 亿个参数。名称中的 B 表示十亿，70B 是参数规模的约数；后文公式中的变量 B 表示批次中的请求数。用 N 表示参数数量， b_w 表示每个权重的存储字节数。BF16 每个权重占 2 字节，公开权重文件中的总字节数为¹³

$$\begin{aligned}
 M_W &= N b_W \\
 &= 70,553,706,496 \times 2 \text{ bytes} \\
 &\approx 141.11 \text{ GB}.
 \end{aligned}$$

本节统一使用十进制 GB，即 $1 \text{ GB} = 10^9 \text{ bytes}$ ；二进制单位 GiB 则为 2^{30} bytes 。因此，141.11 GB 约为 131.42 GiB。设备容量与权重大小换成相同单位后，才能相减。

141.11 GB 权重超过一张 H100 SXM 的名义 80 GB 显存。将权重均分到两张卡上，每张约占 70.55 GB，各剩约 9.45 GB。这里的切分就是让不同设备分别保存并计算模型的一部分。两张卡之间需要交换中间结果，第六章将讨论具体的切分与通信方法。

还可以改变权重的存储精度。量化（**quantization**）用较少的位数表示数值，例如用 8 位整数近似保存原来的 16 位浮点权重，并用缩放因子将整数映射回相应的数值范围。每个被量化的权重由 2 字节减为 1 字节，因而能显著降低内存占用。

本书采用的分组量化方案每 128 个权重共享一个缩放因子，同时让部分参数继续使用 BF16。按真实模型配置计算，8 比特方案的全部权重及量化附加数据约占 73.73 GB，可以放入一张 H100 SXM，留下约 6.27 GB。图 1-7 将两种办法放在相同的容量刻度上：BF16 权重分到两张卡，或者量化后放入一张卡。

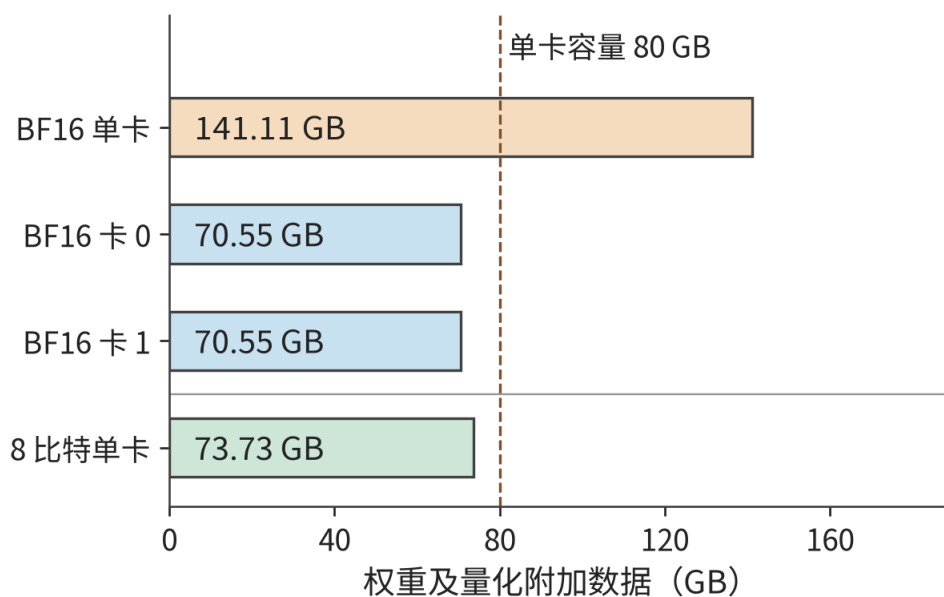


图 1-7 DeepSeek-R1-Distill-Llama-70B: BF16 权重共 141.11 GB，两卡均分后每卡约 70.55 GB；分组 8 比特量化后共 73.73 GB。各条使用相同尺度，虚线表示一张 H100 SXM 的名义 80 GB 容量。柱长只计权重及相应量化附加数据。

KV 缓存如何增加单请求的显存需求 生成文本时，显存还要保存上下文 token 的键和值，称为 **KV 缓存 (key-value cache)**，以便后续计算直接复用；算子执行还需要临时工作区。因此，容量检查应写成

$$M_W + M_{\text{state}} + M_{\text{work}} \leq M_{\text{cap}}.$$

其中 M_W 是权重与量化附加数据， M_{state} 是请求的上下文状态， M_{work} 是工作区， M_{cap} 是设备容量。以该模型的一条 8192-token 请求为例，BF16 KV 缓存为 2.5 GiB，约 2.68 GB；再预留 2 GiB、约 2.15 GB 给工作区，8 比特方案总共约占 $73.73 + 2.68 + 2.15 = 78.56$ GB，符合这一容量预算。第二章将从模型层数与注意力结构推导 KV 大小，并计算更长上下文和更多并发请求的内存需求。

换成名义显存为 24 GB 的 RTX 4090，同一份 73.73 GB 权重就超过了三张卡合计的 72 GB。按相同的名义容量预算，仅保存权重也至少需要四张；还要逐卡检查 KV 和工作区。模型与精度相同，单卡容量不同，所需设备数量就会改变。

显存容量统一单位后，网络传输也要做同样的换算。网络还经常使用 bit/s。8 bits 是 1 byte，所以 400 Gb/s 等于 50 GB/s。按 50 GB/s 的有效带宽传输 1 GB 数据，带宽项为 20 ms。启动时间在每次传输时发生，协议和共享流量则影响应用实际获得的带宽；把它们加入同一条路径，便得到更细的通信时间预算。

练习 1-2 [核心]：容量、位宽与逐卡放置

使用上述真实模型的 BF16 权重 141.11 GB 和 8 比特量化权重 73.73 GB，每张卡容量为 80 GB。为每张卡预留 5 GB 状态与工作区，判断单卡、两卡均分能否容纳模型。再将两卡容量分别改为 48 GB 与 80 GB，判断两种精度各是否可行，并为可行情况给出一种权重分配。最后将 400 Gb/s 换成 GB/s，求传输 1 GB 的理想时间。

只做几步乘除，我们就判断了权重能否放下，并算出了数据传输需要的时间。下面将容量、计算和读取放在一起，估算生成一个 token 的耗时。

1.3 用几个数字估算一次模型执行

1.3.1 生成一个 token，先检查什么

例题 1-1：70B 模型的单步生成如何接近 10 ms 目标？给定一张 H100 SXM，能否让上一节的 DeepSeek-R1-Distill-Llama-70B 在 10 ms 内生成一个新 token？先考虑每参数一字节存储，再比较算力翻倍、带宽翻倍和批内权重复用。

解：先检查显存容量，再计算运算量和读取量。

为便于手算，将该模型参数量近似取为 $N = 70 \times 10^9$ ，先按每参数一字节估算主要权重数据，读入后转换为 BF16 参加矩阵运算。一个请求每步处理一个新 token，每步从显存读取一遍这些权重。

估算条件：权重读取与矩阵计算充分重叠 容量采用上一节计入量化附加数据的 73.73 GB 结果；本节先用 70 GB 近似主要权重的读取量。时间模型只计主要权重矩阵的乘加和一次完整权重读取，采用资源表中的 BF16 矩阵峰值与 HBM 带宽；读取和计算按充分重叠估算。上下文状态访问、格式转换和程序启动在后续模型中逐项加入。这一题设用来识别当前最紧的资源约束。

上一节已经检查过单请求的容量预算。现在看生成速度：权重加载到显存之后，每一步仍需将参与计算的权重送到计算单元。按上述近似，每步读取约 70 GB，下面先计算读取这些数据需要多久。



读取路径：3350 GB/s

$$70 \text{ GB} \div 3350 \text{ GB/s} \approx 20.90 \text{ ms}$$

每参数一字节，每步完整读取一遍

图 1-8 本例按每参数一字节，将主要权重读取量近似取为 70 GB。权重驻留显存，计算单元每步沿同一接口读取一遍；用读取量除以接口带宽，得到 20.90 ms 的读取下界。

同一份权重会在相邻生成步骤间反复使用。对单个请求，下一步的输入由当前输出确定，后续步骤要沿这条依赖逐次推进。权重虽然已经加载到设备，显存到计算单元的读取仍在每步发生。

计算量也可以先做一个近似。对于一次矩阵与向量相乘，一个权重通常参与一次乘法，并累加到结果中；按乘法与加法分别计数，大约对应两次浮点运算。因此，本例的主要矩阵运算量近似为：

$$\begin{aligned}\text{每步矩阵运算量} &\approx 2 \times \text{参数数} \\ &\approx 2 \times 70 \times 10^9 \text{ FLOPs} \\ &= 140 \text{ GFLOPs}.\end{aligned}$$

$2N$ 的来源是在一个投影中，每个权重为一个 token 的特征向量参与一次乘加。参数矩阵中的权重越多，这部分工作越大；一批中本次参与计算的 token 增加时，同一权重矩阵处理更多 token 的特征向量，运算量随之增加。第二章将展开真实模型，在这部分线性工作之外加入随上下文长度变化的注意力交互。

把同时交给一次执行的请求组成一个批次，批内请求数记为 B ，也称 batch size。将批内请求数增至 B ，并让这些请求共同使用一次读入的权重，就得到这个教学模型的三个量：

$$M_W = b_W N, \quad R_W = b_W N, \quad F \approx 2BN.$$

前两个式子在这一步恰好相等，是因为权重保存一份、读取一遍。继续生成下一步时，驻留量仍是 M_W ，却会再增加一次 R_W 的读取。 B 增大则增加本次运算量：同一份权重为更多请求当前输入的 token 完成乘加。第二章将进一步加入各请求独立的上下文状态。

1.3.2 计算与读取分别需要多久

把 $F = 140 \text{ GFLOPs}$ 与 $R_W = 70 \text{ GB}$ 分别除以相应的资源能力，得到单请求的两项时间下界。

$$\begin{aligned}\text{纯权重读取时间} &\geq \frac{70 \text{ GB}}{3350 \text{ GB/s}} \approx 20.90 \text{ ms}, \\ \text{矩阵计算时间} &\geq \frac{140 \text{ GFLOPs}}{989400 \text{ GFLOP/s}} \approx 0.1415 \text{ ms}.\end{aligned}$$

权重读取的时间下界约为矩阵计算的 148 倍。按标称算力，这些矩阵运算很快就能完成；显存却要花更长时间才能把权重送到计算单元。完成当前步骤之前，这些数据必须全部读入。⁴

原因在于，本例每读取一个权重只做一次乘加。计算单元很快处理完已经读入的数据，又要等待后续数据。提高算力只能缩短乘加耗时，无法加快显存读取。

这两项时间应当相加，还是取其中较大者，取决于读取与计算能否重叠。若数据分块到达，计算当前块时可以继续读取下一块；在充分重叠的简化模型中，耗时下界为：

$$T_{\text{step}} \geq T_{\text{lower}} = \max\left(\frac{F}{\Pi}, \frac{R_W}{\beta}\right).$$

进入稳定阶段后，每块数据都要经过读取和计算，较慢的一项决定处理速度。本例中，读取权重远慢于矩阵计算，因此优化应首先针对这 20.90 ms 的读取时间。

讨论：算力、带宽与批内复用分别能缩短多少生成时间？仅权重读取就至少需要 20.90 ms，已经超过 10 ms 的目标。虽然显存能容纳这些数据，读取速度仍然达不到要求。现在做两个最简单的改动。

把计算能力翻倍，矩阵计算项从约 0.1415 ms 变为 0.0707 ms，纯权重读取仍需要约 20.90 ms，决定执行时间的读取下界保持不变。把 HBM 带宽翻倍，读取项才会降为约 10.45 ms，当前下界随之下降。两种改动都使一项资源更强，却没有相同的任务收益。

另一种办法是减少需要读取的数据。在运算量不变的条件下，将权重大小减半，读取下界也降到约 10.45 ms。带宽翻倍让读取速度加倍，权重减半则让待读取的数据减少一半，两种办法在这条算式中取得相同结果。第二、五章将讨论低位宽表示的格式和转换过程。

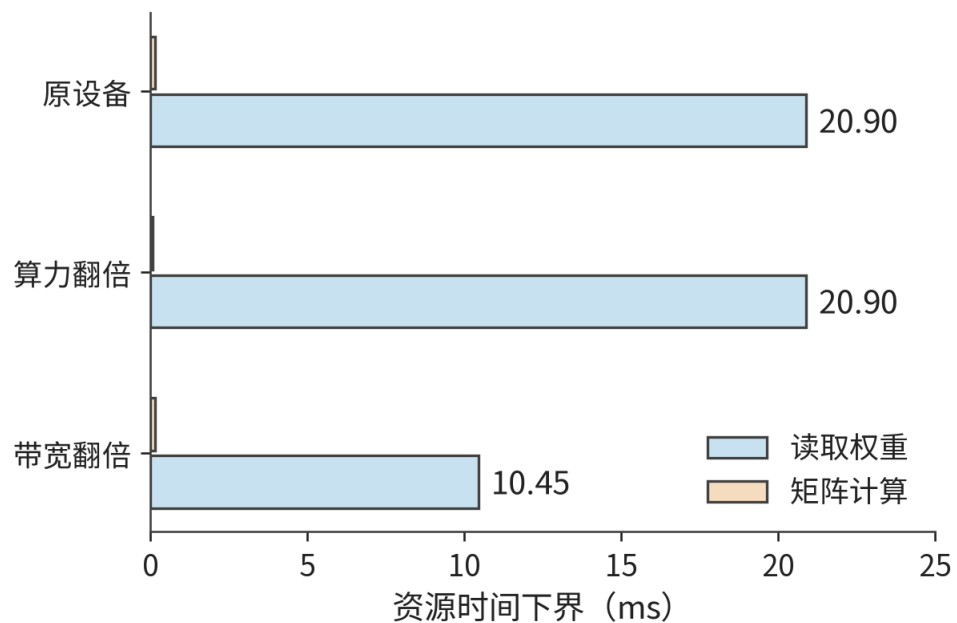


图 1-9 保持运算量与读取量不变，分别把算力或带宽翻倍。蓝条是权重读取下界，橙条是矩阵计算下界；较长的读取项决定这组条件下的优化方向。

接着改变请求组织。假设同时处理 8 个请求，并让它们共享一次权重读取，整批的权重读取仍是 70 GB，矩阵运算量则约为单请求的 8 倍。这时矩阵计算的时间下界约为 1.13 ms，读取下界仍为 20.90 ms。若这一批产生 8 个输出，每个输出分摊的权重服务时间约为 2.61 ms。

整批约 20.90 ms 才完成这些权重的读取，八个请求各得到一个新输出；将它除以八，得到的是每个输出分摊的服务时间。吞吐因而从单请求模型的约 47.9 token/s 提高到约 383 token/s，而每个请求仍经历整批执行。批内复用增加同一时间内生成的输出数，单请求延迟则取决于它经历的执行与等待。

同一份权重，为八个请求服务

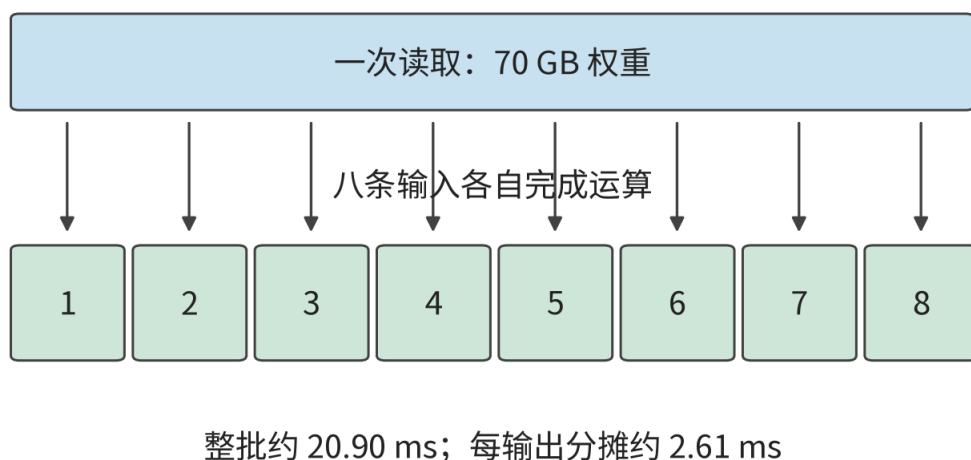


图 1-10 一批八个请求共享一次权重读取，各产生一个输出。整批读取仍需约 20.90 ms，除以八得到每输出分摊的服务时间；每个请求经历整批执行。

batch 增加到一定程度后，计算时间会赶上权重读取时间。令 $2BN/\Pi = b_W N/\beta$ ，得到转折点

$$B_* = \frac{b_W \Pi}{2\beta}.$$

本例取 $b_W = 1$ 、 $\Pi = 989.4 \times 10^{12}$ FLOP/s、 $\beta = 3.35 \times 10^{12}$ bytes/s，得到 $B_* \approx 147.7$ 。在这个只计主要矩阵与权重读取的模型中，batch 较小时，增加请求可以分摊权重读取开销；超过约 148 后，计算成为较长的一项，继续增加 batch 会近似按比例增加整批时间。

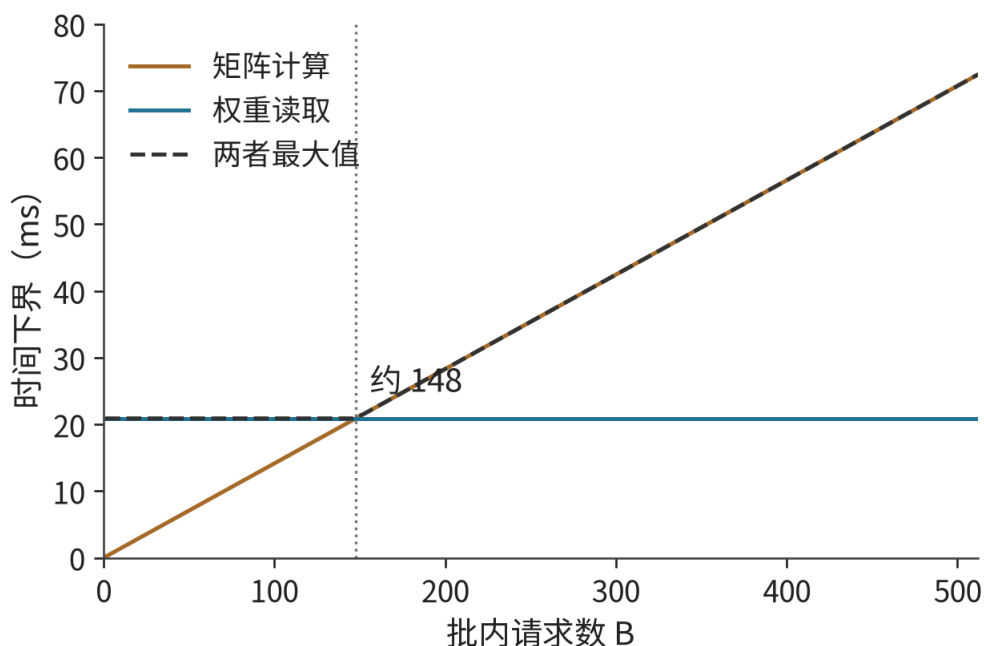


图 1-11 批内请求增加时，矩阵运算量按 $2BN$ 增长，权重读取保持每批 70 GB。约 148 个请求处两项下界相等，随后计算项主导。

把每批输出数 B 除以上图中的时间下界，便得到这组题设对应的吞吐上界。转折前，共享读取被更多输出分摊；转折后，计算时间与输出数一起增长，曲线逐渐进入平台。

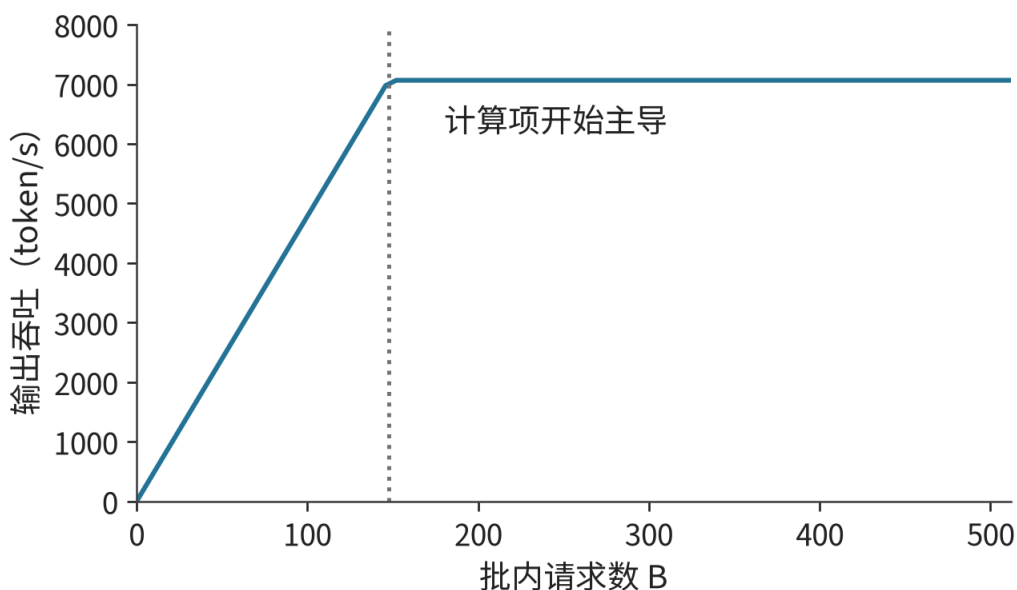


图 1-12 同一批量模型的理想输出吞吐。每批输出数除以时间下界得到曲线；竖虚线与前图对应同一个约 148 请求的转折点。

同一转折也可以用算术强度 $I = F/R_W = 2B/b_W$ 描述，即每读取一字节数据所完成的运算次数。当 I 小于设备的算力与带宽之比 Π/β 时，读取较慢；超过这个比值后，计算较慢。这就是第四章 Roofline 模型的出发点。⁵

练习 1-3 [核心]：改变资源能否满足时限

沿例题 1-1，将有效带宽设为标称值的 70%，计算吞吐设为峰值的 50%。分别取 $B = 1, 16, 64$ ，计算整批时间下界、每输出分摊时间与理想吞吐。目标是每请求每步不超过 10 ms：哪些 batch 可被下界直接排除？再把权重读取量减半、运算量保持不变，重新判断。求计算与读取相等时的 B_* ，并解释为什么增大 batch 不能无限降低每请求延迟。

1.3.3 用实测检验估算

前面的模型预测了两个趋势：batch 较小时，吞吐随 batch 增加；整批时间则受到一次权重读取的限制。将模型用于真实程序时，还要检查各请求新增的上下文访问和计算。图 1-13、1-14 给出 Qwen3-8B 在 RTX PRO 6000 Blackwell 上的测量：使用 BF16 权重。vLLM 是加州大学伯克利分校等机构的研究者于 2023 年提出的开源推理服务系统，最初重点解决 KV 缓存浪费显存、限制批量的问题；它组织模型请求并执行推理。本次采用 0.23 版本的 eager 模式，即按程序运行过程逐项提交设备工作。每请求输入 2048 个 token、生成 256 个 token，每档 batch 测量三次。⁶ 吞吐表示单位时间内产生的输出数；TPOT (time per output token) 在这里表示客户端观察到的平均输出间隔。下面先看整批产出，再看单个请求的生成节奏。

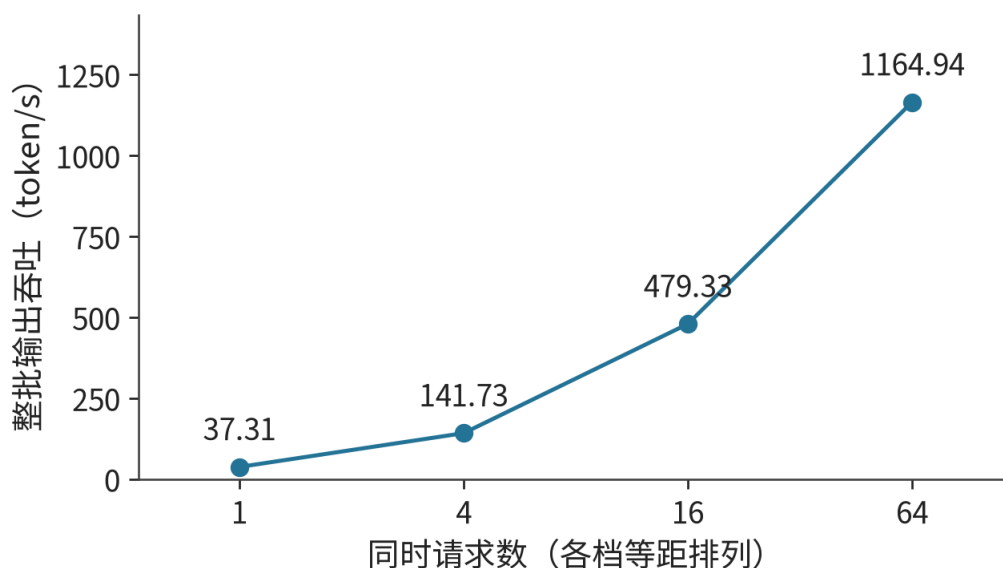


图 1-13 Qwen3-8B 实测的整批输出吞吐。四档请求数等距排列，纵轴从零开始；模型、精度、输入输出长度和计时范围保持一致。

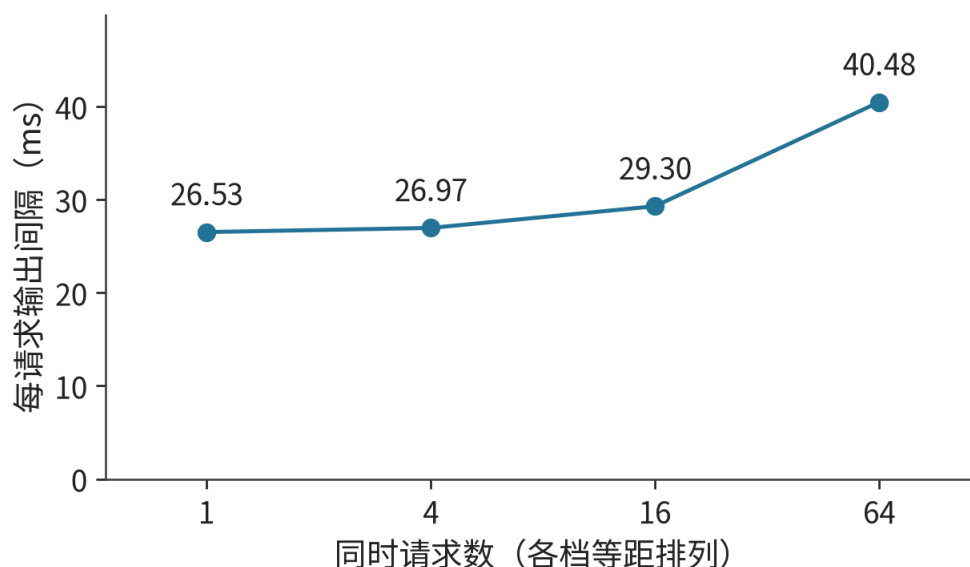


图 1-14 同一组实测中的每请求输出间隔。吞吐增加的同时，单个请求的平均间隔也在增大；两张图分别回答设备产出和用户等待的问题。

从 1 个请求增到 64 个请求，整批吞吐约提高 31 倍，单请求输出间隔则从约 26.5 ms 增至 40.5 ms。两幅图合起来描述批量组织的取舍：设备在同一时间内完成更多请求的工作，每个请求也经历更长的一步执行。

图 1-13 的吞吐按整批输出数除以处理这批请求的总时间计算，其中包括输入处理时间。TPOT 则先对每个请求计算首末输出事件之间的时间，再除以剩余的 255 个输出间隔，最后求所有请求的平均值。

前者描述整个实例的产出，后者描述单个用户经历的生成节奏。第三章将进一步分析请求到达、排队和少量慢请求对这些指标的影响。

这一趋势可以从权重与上下文状态的区别来解释。多个请求共同使用模型权重，但各自保存上下文状态。并发增加时，矩阵一次处理更多请求当前输入的 token 表示，权重得到更多复用；与此同时，注意力计算和上下文访问也随请求数增加。

因此，测量对基本模型作出的修正是：每批共享权重读取之外，还存在随请求数增长的工作。可以先把一步耗时写成

$$T_{\text{step}} \approx \max\left(\frac{BF_1}{\Pi}, \frac{R_W + BR_{\text{state},1}}{\beta}\right),$$

其中 F_1 是每请求的运算量， $R_{\text{state},1}$ 是每请求的状态读写量。batch 增加时，共享项被更多输出分摊，独立状态项却同步增加。下一章将从实际模型算出这些项，并预测上下文变长后转折会怎样移动。

练习 1-4 [核心]：用测量检验预测

根据图 1-13、1-14 标出的四档并发请求测量值，计算相对单请求的吞吐加速比和 TPOT 增长比例。如果只有共享权重读取、没有新增计算和上下文访问，吞吐应怎样变化？对照实际结果，提出两个可区分的原因，并设计一次只改变上下文长度的测量来判断它们。指出测量前需要保持不变的模型、输出长度与计时起止点。

回看这次估算，我们实际上一直在回答五个问题：搬什么、搬多少、搬几次、经过哪里、谁在等待。模型结构决定需要哪些权重，位宽和参数量决定存储大小，批量复用改变了读取次数，显存接口提供了带宽，执行依赖决定了等待。这五个问题把模型结构、数据路径和执行顺序连成一份可以逐项计算的预算。

1.4 需求如何推动架构设计

架构设计从需要解决的问题出发。不同年代的业务规模、设备能力与软件要求，决定了设计者首先需要改变什么。下面依次看 Google 在 2013 年面对的语音推理需求、微软在 2015—2016 年扩展 Azure 网络的需求，以及华为在大模型发展过程中组织多设备计算与存储的需求。

1.4.1 TPU

TPU 是 Google 为神经网络计算设计的张量处理器，其第一代产品记为 TPU v1。介绍这代产品的论文回顾了一次需求判断。Google 很早就讨论过在数据中心使用 GPU、可在部署后配置硬件逻辑的 FPGA，或专用芯片，但在一些早期应用中，既有数据中心的剩余能力已经能够承接需求。2013 年，一项语音搜索使用量预测改变了讨论：如果每天使用三分钟语音搜索，满足这项计算需求可能需要让数据中心规模翻倍。Google 因而推进专用推理芯片设计，TPU v1 于 2015 年进入数据中心部署，架构论文于 2017 年发表。²

把每天的请求量记为 Q 、每请求需要的计算量记为 F_1 ，每日总需求就是 QF_1 。请求量增加，现有设备的空闲能力最终会被用尽，此时可以增加通用服务器，也可以开发专用处理器。后者有前期开发成本，却可能降低每请求的设备时间和能耗；服务规模越大，累计节省的成本就越容易抵偿前期投入。

专用处理器可以为反复出现的矩阵运算配置更多计算单元，再用片上缓冲和数据通路为它们提供输入。响应时间与功耗要求决定了应如何分配计算、存储和通信资源。这样，应用规模的变化就落实到了芯片设计上。

增加专用计算与数据搬运资源

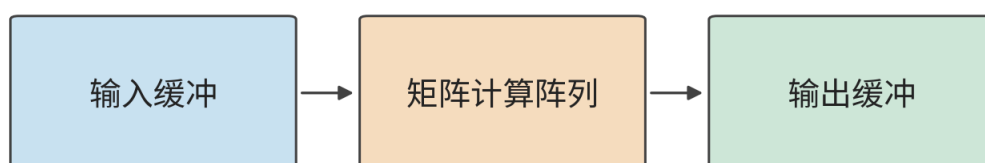


图 1-15 专用处理器围绕反复出现的矩阵运算组织计算阵列和输入输出缓冲。缓冲是临时保存待计算或已算完数据的存储区域；三个方框及其箭头展示数据搬运方向。

第 4 章将介绍 TPU 的矩阵阵列、缓冲和数据通路。本例先说明一个关系：应用的使用量达到一定规模后，专用处理器就可能比通用处理器更合算。

1.4.2 SmartNIC

微软 Azure 在 2015 年设计新一代 40 Gb/s 云服务器时，面临另一类增长：网络带宽提高，虚拟网络策略也越来越复杂。**SmartNIC**（智能网卡）是能够在网卡上执行部分数据处理的设备。微软从 2015 年末起在新部署的 Azure 服务器中安装基于 FPGA 的 SmartNIC，2016 年向客户提供 Accelerated Networking（加速网络）服务。FPGA 是可通过编程配置硬件逻辑的芯片，使网络处理能够随策略更新而改变。¹⁴

这项设计要同时满足两个要求：把更多 CPU 核留给租户的应用，又能快速更新网络隔离、转发和访问控制规则。云服务器上的网络处理就是一个例子：CPU 除了运行应用，还要处理收发的数据包。虚拟机是在一台物理主机上划分出的独立计算环境。多个虚拟机共用物理网络时，平台需要识别数据的归属、查找转发规则、完成封装，并维持隔离。

先估算这部分工作占用多少 CPU。40 Gb/s 链路在最小帧条件下每秒约需处理六千万个包。若一个核每秒处理一千万至两千万个包，持续转发所需核数约为

$$n_{\text{core}} = \frac{\lambda_{\text{packet}}}{\mu_{\text{core}}} \approx 3\text{—}6.$$

其中 λ_{packet} 为包到达率， μ_{core} 为单核处理率。这些核反复完成相同的包处理流程，将相应工作移到网卡，可以把 CPU 时间留给应用。⁸

Azure 的做法是让主机软件管理复杂策略，把适合重复执行的包处理规则交给 FPGA。数据经过网卡时就完成这些处理，CPU 可以把更多时间用于客户应用。设计的关键是同时保留软件更新能力与硬件处理速度。

把包处理放到数据经过的位置

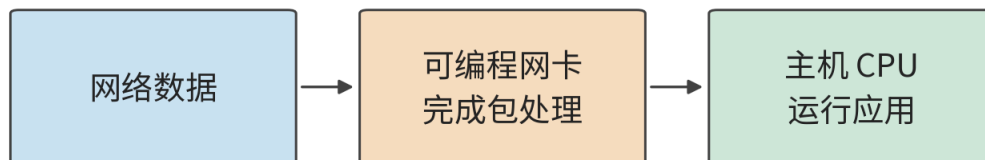


图 1-16 可编程网卡在数据进入主机前完成指定的包处理。图中实线跟踪数据；网卡承担的处理减少主机 CPU 的辅助工作。

将包处理移到网卡后，部分操作可以在数据到达时直接完成，CPU 因而少做一些工作。如果网卡还需要主机内存中的数据，就要通过 PCIe 读取；此时，返回延迟和可同时发出的请求数决定了读取速度。第 7 章将用 KV-Direct 分析这种访问；它是让可编程网卡直接处理键值存储请求的系统。键值存储按唯一的键查找、读取或更新对应的值。

1.4.3 Unified Bus

SmartNIC 改变了单台服务器内部的分工。若问题是单台设备的资源总量不足，就需要进一步考虑设备之间怎样协作。当一张设备容纳不下模型和运行状态时，最直接的办法之一是增加设备。但“总容量变大”与“任务能够高效执行”之间，还隔着工作划分、数据交换和同步。

例如，两张卡的显存总量可能足够保存权重，但模型执行所需的数据必须落在能够使用它的设备上。若一步计算依赖另一张卡的结果，就要等待交接；若多张卡共同完成一个操作，还要组织相应的协作。设备数增加带来了资源，也增加了连接这些资源的工作。

华为面向昇腾等异构计算设备推进的 **UB (Unified Bus, 统一互联)**，把问题扩大到多设备协作。根据项目参与者的回顾，相关研究在 OpenAI 于 2020 年发布自回归语言模型 GPT-3 之前已经开始；GPT-3 展现出大模型的能力后，多设备协同需求获得更广泛的认可，项目投入随之扩大。⁹

这一时期的核心需求是让越来越大的模型使用多张加速卡，并让计算设备更方便地访问其他设备上的内存和数据。UB 因而尝试统一设备之间的访问与通信机制。统一互联希望降低不同设备之间访问和交接的代价，使上层可以采用更灵活的资源组织。统一的访问机制使设备能够直接使用更大范围内的资源，而拓扑决定这些访问的距离与带宽。模型分工与互联设计由此紧密相连。

让多台设备直接交换所需数据

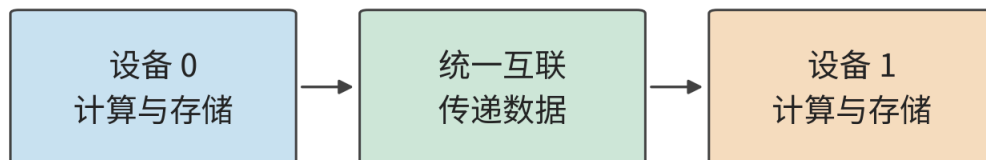


图 1-17 统一互联连接不同设备的计算与存储资源。模型分工确定要交换什么，互联负责把数据送到后续使用它的设备。

从语音推理到云网络，再到大模型的多设备执行，设计目标依次落在计算效率、主机 CPU 占用和跨设备访问上。需求改变了需要优先解决的瓶颈，也改变了硬件、软件与互联之间的分工。

再看开篇的六层图，三个案例分别从不同问题出发，却都会改变计算与数据移动的组织。应用需求推动资源专用化，处理位置改变主机与网卡之间的工作分配，更大的模型则要求重新安排设备协作。后面的章节会沿这些具体关系展开本书的主线：数据搬运塑造了 **AI Infra** 的架构。

回到两卡模型：权重均分可以满足逐卡容量约束，但每个阶段还要等待输入数据和前序结果。增加设备改变了 M_{cap} 和可用算力，也引入新的通信量与依赖。因此，第一步先检查每台设备能否放下所需数据，第二步计算各设备的运算量与读写量，第三步根据执行依赖计算总时间。模型再大、设备再多，这个分析顺序仍然成立。

谬误与陷阱

峰值算力提高一倍，程序就会快一倍。先检查计算是否限制当前执行。例题 1-1 的读取项远大于计算项，提高矩阵峰值几乎不改变该下界；若优化的是完整任务的一部分，还要用 Amdahl 定律检查收益上限。

权重放得下，模型就能够按目标并发运行。权重只是常驻数据的一部分。上下文状态、工作区和运行时预留共同消耗容量，多设备还必须逐卡核算。

吞吐提高，每个用户就更快。批内复用减少单位输出分摊的读取，但每个请求仍经历整批执行和自己的排队。吞吐与响应时间应同时报告。

本章数字速查

下面用前面实测采用的 Qwen3-8B 汇总模型侧的关键数字。**Prefill**（输入处理）一次处理已有输入，并得到首个输出；**Decode**（逐步生成）每次把一个新 token 送入模型，利用已经保存的 KV 继续生成。本表固定 BF16 权重、激活和 KV，并发为 1；prefill 的输入为 2048 个 token，decode 则在已有 2048-token 上下文后执行一步。¹⁶

模型需求	数值	如何使用
完整 BF16 权重	16.38 GB	判断模型加载所需的显存
每个上下文 token 的全模型 KV	144 KiB	上下文长度乘以此数，得到单请求 KV 容量
2048-token 上下文的 KV	288 MiB	约 0.302 GB，随独立请求数增加
2048-token prefill 的矩阵运算	29.69 TFLOPs	除以相应矩阵算力，得到计算时间下界
一步 decode 的矩阵运算	16.34 GFLOPs	输入为一个 token，另访问已有上下文
一步 decode 的主要权重读取	15.14 GB	权重读一次；嵌入层只读取当前 token 对应的一行
一步 decode 的旧 KV 读取	0.302 GB	每份上下文 KV 读一次时的字节数

用计算量与读取量估算执行时间下界 先单独计算矩阵运算和数据读取：按 H100 的 989.4 TFLOP/s 矩阵峰值换算，prefill 的矩阵运算约需 30.0 ms，decode 约需 0.0165 ms；按 3.35 TB/s 显存带宽，一步 decode 的主要权重与旧 KV 读取约需 4.61 ms。这里分别给出资源时间下界，完整执行还要沿算子顺序加入其他运算、实际访存和启动时间。第二至第五章将逐项建立这些计算。

这组数字解释了两阶段的差别：prefill 在一次调用中处理大量输入，矩阵工作量较大；单请求 decode 每次只处理一个位置，却仍要读取大量权重和上下文。提高计算吞吐、提高带宽和增加批内复用，分别作用于不同的时间项。

需要查实测响应时，回到图 1-13、1-14：那里使用 RTX PRO 6000 Blackwell、vLLM 0.23 和明确的请求长度，显示吞吐与客户端输出间隔。GPU 规格表提供硬件性能数据，模型表用于估算工作量，实测图则展示系统实际运行的表现。

-
1. “十八层宝塔”的提出者为海思首席科学家廖恒博士。全栈讨论的背景资料见 [《Unified Bus 背后的思考》本地原件及系统抽象边界取材记录](#)。六层图为本书自绘的教学分组。↩
 2. Jeff Dean, LADIS 2009 演讲, [本地 PDF](#), 查阅“Numbers Everyone Should Know”与随后纸笔估算的幻灯片。正文 20 次寻道的例子是本书教学假设。↩
 3. NVIDIA, [H100 架构白皮书与规格页面快照](#)。本章采用 SXM 形态与 BF16 稠密矩阵运算规格；原始输入版本和校验值见[配图来源清单](#)。↩
 4. [70B 近似预算复算及配套场景](#)；[单位与逐卡容量复算](#)。1.3 的时间估算图读取这组固定的 70B 近似结果；1.2.3 的容量图使用真实权重索引与分组量化结果。↩
 5. Williams、Waterman、Patterson, [Roofline 论文](#)。本章只引入算术强度和资源上界的直觉，实际层次、访问量和执行效率在第 4、5 章展开。↩
 6. [练习 1-4 记录说明](#)、[结构化分析结果及后续补测](#)。正文和图 1-13、图 1-14 采用最初记录的短输入组；更长输入与缓存条件不同，未合并作长度因果比较。↩
 7. Jouppi 等, [TPU v1 论文](#)，第 2 节关于起源、架构与实现的说明。三分钟语音搜索是论文记载的历史需求预测，不是本书当代负载测量。↩
 8. [作者博士论文](#)，第 4.2.1 节简单转发基线；[核数预算复算](#)。约六千万包每秒来自 40 Gb/s 与每帧占用 84 字节线时的计算，3 至 6 核未包含完整虚拟化工作。↩
 9. 项目参与者回顾：[《Unified Bus 背后的思考》](#)，“UB 超节点”一节记述研究先于 GPT-3、2020 年后扩大投入的过程。↩

10. 这一对照是本书的分析视角。背景与适用边界见[抽象边界调研](#)，其中保存了 OSDI / SOSP、基础模型报告与隔离机制的来源；基础模型承接多种下游任务的讨论见 [Stanford CRFM](#)。↩
11. 请求图是通用职责示意；调度与状态管理可参照 [vLLM Scheduler 文档](#)。路由策略、tokenizer 位置以及 prefill / decode 是否分离属于部署选择。↩
12. 具体产品例子见 [NVIDIA GB200 NVL72](#)、[硬件指南](#)与[网络指南](#)。图 1-5 使用一般化连接，不是该产品的复刻拓扑。↩
13. 参数与 BF16 字节数来自 [DeepSeek-R1-Distill-Llama-70B 公开权重索引](#)及[固定版本配置](#)。分组量化的权重总量采用[逐项计算记录](#)中的全模型汇总，每组 128 个参数、缩放因子占 2 字节；该记录的多卡布局不作为本节单卡运行配置。RTX 4090 的 24 GB 名义规格见[官方页面存档](#)。本节用规格标称 GB 作统一容量预算。↩
14. Firestone 等，微软，[Azure Accelerated Networking: SmartNICs in the Public Cloud](#)，NSDI 2018；[原论文文本](#)。摘要给出 2015 年末部署、2016 年向客户提供服务的时间，第 3 节说明保留 CPU、可编程性与带宽扩展的设计目标。↩
15. 数字取自[固定 GPU 规格与逐项出处](#)，对应 rtx4090、a100-80gb-sxm、h100-sxm；各精度、累加方式与稠密条件分别核对。GB 与 TB 在本表均为十进制单位。↩
16. [速查数字复算源码](#)根据固定 Qwen3-8B 配置统计 prefill 与 decode 的矩阵运算，读取量来自[批内权重复用计算](#)。权重占用包含完整嵌入表，逐步生成时只读取当前 token 对应的嵌入行；读写时间采用每份主要数据访问一次的理想条件。↩
17. [DeepSeek V4.1 官方技术报告](#)，第 1、2、3 节与第 6 节；[跨章会话的固定条件与复算](#)。架构、格式和部署机制按报告区分，教学时间假设不作实测。↩

本章小结

应用通过模型与上下文表达行为，使许多任务共享相近的底层计算。掌握模型的计算关系与状态寿命，就能统筹考虑原来分属不同层次的设计选择：改变数据表示、合并执行步骤、复用准备工作，或缩短设备交接。后续各章将量化这些改动的收益，说明哪些设计取舍因此改变，再根据新的约束选择模型与系统方案。

分析系统时，先检查数据能否放下，再用运算量和读写量估算耗时，最后根据执行顺序分析重叠与等待。在 70B 算例中，权重读取比矩阵计算慢约 148 倍，因此增加带宽、减少读取和提高复用更有助于

缩短时间。批内共享可以提高吞吐，而每个请求的响应时间还要包括整批执行和排队。

本章核心练习为 1-2、1-3、1-4：分别练习单位与可行性、改变条件后的预算，以及预测与观察的对照。下一章将根据真实模型配置，细化 $2N$ 计算量与权重读取量的估算，为这些计算提供具体的矩阵尺寸与状态大小。